
CREACIÓN DE MAPA CONCEPTUAL A PARTIR DE UN TEXTO (HIPÓNIMOS/HIPERÓNIMOS)

Minería de texto

Dra. Ana Bertha Ríos Alvarado

José Eliuth Torres Ortiz

INTRODUCCIÓN

- El objetivo de este proyecto es crear un sistema de análisis de texto que identifique relaciones de **hiperónimos e hiponimos** utilizando **expresiones regulares**, análisis semántico y dependencias gramaticales. Este sistema genera mapas conceptuales a partir del texto procesado.

FUNCIONES

❑ Preprocesamiento del texto.

- Limpieza de texto como quitar signos, puntuación, palabras vacías, convertir a minúsculas el texto.

❑ Extracción de relaciones de hiperónimos e hipónimos.

- Expresiones regulares : Extracción de patrones X es un tipo de Y.
 - Análisis semántico con **Wordnet**.
 - Análisis de dependencia: con **SpaCy** se extraen las relaciones basado en dependencias gramaticales.
-

FUNCIONES

❑ Generación de mapas conceptuales.

➤ Con la librería de networkx y matplotlib.

❑ Generación de código Mermaid.

➤ Permite usar herramientas web online para ver el mapa conceptual creado.

❑ Interfaz de usuario interactiva.

➤ Con la librería **streamlit** para crear interfaz de usuario interactiva y visualizar los resultados.

IMPORTACIONES DE LIBRERÍAS

- **nltk**: Para trabajar con WordNet y obtener relaciones semánticas (hiperonimia/hiponimia).
 - **networkx y matplotlib**: Para generar y visualizar los mapas conceptuales.
 - **re** (expresiones regulares): Para buscar patrones en el texto.
 - **streamlit**: Para crear la interfaz de usuario interactiva.
 - **spaCy**: tokenización, extracción de relaciones.
 - **Pillow**: manejo de imágenes.
-

DESCARGA DE RECURSOS

- **`nltk.download('wordnet')`**: para trabajar con WordNet y obtener los hiperónimos e hipónimos.
- **Modelo de SpaCy**: para realizar la tokenización y análisis de dependencias.
(`en_core_web_sm`)

CÓDIGO

- Extracción de sustantivos y nombres propios
- Elimina stopwords.
- Devuelve una lista con lemas (formas base) de esas palabras, en minúsculas, sin duplicados.

```
def clean_hyponyms(hyponyms_text):  
    doc = nlp(hyponyms_text)  
    clean = []  
    for token in doc:  
        if token.pos_ in ["NOUN", "PROPN"]:  
            if token.text.lower() not in custom_stopwords:  
                clean.append(token.lemma_.lower())  
    return list(set(clean))
```

CÓDIGO

Se utilizan 14 expresiones regulares que identifican los hiperónimos e hipónimos, como:

- X is a hypernym of Y
- X such as Y
- Y categorized as X

```
patterns = [  
    r"(?P<hypernym>\w+)\s+is\s+a\s+hypernym\s+of\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hypernym>\w+)\s+such\s+as\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hyponyms>[\w\s,]+)\s+is\s+a\s+type\s+of\s+(?P<hypernym>\w+)",  
    r"(?P<hypernym>\w+)\s+including\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hyponyms>[\w\s,]+)\s+is\s+a\s+kind\s+of\s+(?P<hypernym>\w+)",  
    r"(?P<hypernym>\w+)\s+consists\s+of\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hypernym>\w+)\s+and\s+other\s+(?P<hyponyms>\w+)",  
    r"(?P<hypernym>\w+)\s+or\s+similar\s+(?P<hyponyms>\w+)",  
    r"(?P<hyponyms>\w+)\s+belong\s+to\s+the\s+category\s+of\s+(?P<hypernym>\w+)",  
    r"(?P<hyponyms>\w+)\s+categorized\s+as\s+(?P<hypernym>\w+)",  
    r"examples\s+of\s+(?P<hypernym>\w+)\s+include\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hypernym>\w+)\s+divided\s+into\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hypernym>\w+)\s+characterized\s+by\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hypernym>\w+)\s+comprising\s+(?P<hyponyms>[\w\s,]+)",  
    r"(?P<hypernym>\w+)\s+like\s+(?P<hyponyms>[\w\s,]+)"  
]
```

CÓDIGO

Analiza el texto de entrada y busca los patrones específicos, basado en las expresiones regulares establecidas.

Extrae las palabras y las guarda como par (lista de tuplas) y al final devuelve una lista con toda la lista de palabras encontradas.

```
def extract_hyponym_patterns(text):  
    pairs = []  
    for pattern in patterns:  
        matches = re.finditer(pattern, text, re.IGNORECASE)  
        for match in matches:  
            hyper = match.group("hypernym").strip().lower()  
            hypos_raw = match.group("hyponyms")  
            hypos_clean = clean_hyponyms(hypos_raw)  
            for hypo in hypos_clean:  
                pairs.append((hyper, hypo))  
    return pairs
```

CÓDIGO

Se genera un diagrama utilizando Mermaid para representar un mapa conceptual.

A partir de las relaciones de hiperónimos e hipónimos extraídas

```
def generate_mermaid_code(grouped_pairs):
    mermaid_code = "graph TD\n"
    mermaid_code += "    Conceptual_Map\n"

    # Agregar relaciones entre Conceptual Map y cada hiperónimo
    for hypernym, hyponyms in grouped_pairs.items():
        mermaid_code += f"    Conceptual_Map --> {hypernym}\n"
        for hypo in hyponyms:
            mermaid_code += f"    {hypernym} --> {hypo}\n" #

    return mermaid_code
```

CÓDIGO

En esta parte del código se utiliza la librería networkx para la creación de los nodos, con las relaciones obtenidas.

```
def build_and_save_concept_maps(grouped_pairs):
    root_node = "Conceptual Map"
    global_graph = nx.DiGraph()

    saved_images = []

    # Crear cada relación individualmente y agregar hip
    for hypernym, hyponyms in grouped_pairs.items():
        G = nx.DiGraph()
        for hypo in hyponyms:
            G.add_edge(hypernym, hypo)
            #
            global_graph.add_edge(root_node, hypernym)
            global_graph.add_edge(hypernym, hypo)
```

CÓDIGO

Aquí simplemente guardamos la colección creada entre nodos, como una imagen usando la librería PILLOW.

```
# Guardar la imagen del mapa conceptual individual
plt.figure(figsize=(8, 6))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_size=2000, font_size=10, arrows=True)
plt.title(f"Concept Map: {hypernym}", fontsize=14)
filename = f"concept_map_{hypernym}.png"
plt.savefig(filename)
plt.close()
saved_images.append(filename)
```

CÓDIGO

A continuación se explicara el código utilizado para la interfaz con la Libreria **Streamlit**.

```
def main():  
    st.title("🧠 Concept Map Generator (Text Mining using Regular Expressions)")  
  
    input_text = st.text_area("👉 Paste or type your text here:", height=200)  
  
    if st.button("📌 Generate Concept Maps"):  
        if not input_text.strip():  
            st.warning("Please enter a text to analyze.")  
            return
```

CÓDIGO

- Limpieza del texto,
- Extracción de las relaciones de hiponimia e hiperonimia
- Agrupación de relaciones con defaultdict.
- Visualización de las relaciones encontradas.

```
# Mostrar texto después de preprocesamiento
cleaned_text = ' '.join(clean_hyponyms(input_text))
st.subheader("🔪 Processed Text (after cleaning):")
st.write(cleaned_text) # Muestra el texto limpio debajo del cuadro de texto

pairs = extract_hyponym_patterns(input_text)

if not pairs:
    st.info("No hypernym/hyponym relationships found.")
else:
    grouped = defaultdict(list)
    for hyper, hypo in pairs:
        grouped[hyper].append(hypo)

    st.subheader(f"🔍 Relationships found: {len(grouped)}")
    for hyper, hypos in grouped.items():
        st.markdown(f"**{hyper.capitalize()}** → {' '.join(hypos)}")

    st.subheader("🗺️ Concept Maps Generated:")
    saved_files = build_and_save_concept_maps(grouped)
    for filename in saved_files:
        if os.path.exists(filename):
            st.image(Image.open(filename), caption=filename, use_container_width=True)
```

CÓDIGO

Se utiliza la función para generar el código mermaid.

Por ultimo se muestra el enlace para visualizar el mapa conceptual con el código mermaid.

```
# Generate Mermaid code
mermaid_code = generate_mermaid_code(grouped)
st.subheader("📄 Mermaid Code of the Concept Map:")
st.code(mermaid_code, language="mermaid")

# Add link to Mermaid viewer
mermaid_viewer_url = "https://mermaid-js.github.io/mermaid
st.markdown(f"[View the concept map in the Mermaid viewer

== " main "
```

CONCLUSIONES

- Con las diferentes estrategias se logro generar mapas conceptuales a través de un texto de entrada, utilizando diferentes enfoques como expresiones regulares, semántica, análisis de dependencia.
 - El sistema es efectivo para identificar relaciones de hiperonimia e hiponimia en textos, aunque existen áreas de mejora en cuanto a la cohesión y optimización del proceso.
-